PYTHON PROGRAMMING

LECTURE

NOTES

DEPARTMENT OF COMPUTER SCIENCE AND I T

PYTHON PROGRAMMING

UNIT – I

INTRODUCTION DATA, EXPRESSIONS, STATEMENTS

Introduction to Python and installation, data types: Int, float, Boolean, string, and list; variables, expressions, statements, precedence of operators, comments; modules, functions-- - function and its use, flow of execution, parameters and arguments.

Introduction to Python and installation:

Python is a widely used general-purpose, high level programming language. It was initially designed by **Guido van Rossum in 1991** and developed by Python Software Foundation. It was mainly developed for emphasis on code readability, and its syntax allows programmers to express concepts in fewer lines of code.
Python is a programming language that lets you work quickly and integrate systems more efficiently.

There are two major Python versions- **Python 2 and Python 3**.

• On 16 October 2000, Python 2.0 was released with many new features. • On 3rd December 2008, Python 3.0 was released with more testing and includes new features.

Beginning with Python programming:

1) Finding an Interpreter:

Before we start Python programming, we need to have an interpreter to interpret and run our programs. There are certain online interpreters like **https://ide.geeksforgeeks.org/**, http://ideone.com/ or http://codepad.org/ that can be used to start Python without installing an interpreter.

***Windows*:** There are many interpreters available freely to run Python scripts like IDLE (Integrated Development Environment) which is installed when you install the python software from **http://python.org/downloads/**

2) Writing first program:

```
# Script Begins

     Statement1

     Statement2

     Statement3

# Script Ends
```


Differences between scripting language and programming language:

SCRIPTING LANGUAGE	PROGRAMMING LANGUAGE
A programming language that supports scripts: programs written for a special run-time environment that automate the execution of tasks	A formal language, which comprises a set of instructions used to produce various kinds of output
Execution speed is slow	Compiler-based languages are executed much faster while interpreter-based languages are executed slower
Can be divided into client-side scripting languages and server-side scripting languages	Can be divided into high-level, low-level languages or compiler-based or interpreter-based languages
Easier to learn	Not as easy to learn
Ex: JavaScript, Perl, PHP, Python and Ruby	Ex: C, C++, and Assembly
Mostly used for web development	Used to develop various applications such as desktop, web, mobile, etc.

Why to use Python:

The following are the primary factors to use python in day-to-day life:

1. **Python is object-oriented**
   Structure supports such concepts as polymorphism, operation overloading and multiple inheritance.
2. **Indentation**
   Indentation is one of the greatest feature in python

3. **It's free (open source)**
   Downloading python and installing python is free and easy
4. **It's Powerful**
   - Dynamic typing
   - Built-in types and tools
   - Library utilities

• Third party utilities (e.g. Numeric, NumPy, sciPy)

• Automatic memory management

5. It's Portable

• Python runs virtually every major platform used today

• As long as you have a compaitable python interpreter installed, python programs will run in exactly the same manner, irrespective of platform. **6. It's easy to use and learn**

• No intermediate compile

• Python Programs are compiled automatically to an intermediate form called byte code, which the interpreter then reads.

• This gives python the development speed of an interpreter without the performance loss inherent in purely interpreted languages.

• Structure and syntax are pretty intuitive and easy to grasp.

7. Interpreted Language

Python is processed at runtime by python Interpreter

8. Interactive Programming Language

Users can interact with the python interpreter directly for writing the programs **9. Straight forward syntax**

The formation of python syntax is simple and straight forward which also makes it popular.

Installation:

There are many interpreters available freely to run Python scripts like IDLE (Integrated Development Environment) which is installed when you install the python software from http://python.org/downloads/

Steps to be followed and remembered:

Step 1: Select Version of Python to Install.

Step 2: Download Python Executable Installer.

Step 3: Run Executable Installer.

Step 4: Verify Python Was Installed On Windows.

Step 5: Verify Pip Was Installed.

Step 6: Add Python Path to Environment Variables (Optional)



Working with Python

Python Code Execution:

Python's traditional runtime execution model: Source code you type is translated to byte code, which is then run by the Python Virtual Machine (PVM). Your code is automatically compiled, but then it is interpreted.

Source Byte code Runtime

m.py m.pyc **PVM**

Source code extension is .py
Byte code extension is .pyc (Compiled python code)

There are two modes for using the Python interpreter:

- Interactive Mode
- Script Mode

Running Python in interactive mode:

Without passing python script file to the interpreter, directly execute code to Python prompt. Once you're inside the python interpreter, then you can start.

>>> print("hello world")

hello world

Relevant output is displayed on subsequent lines without the >>>

symbol >>> x=[0,1,2]

Quantities stored in memory are not displayed by default.

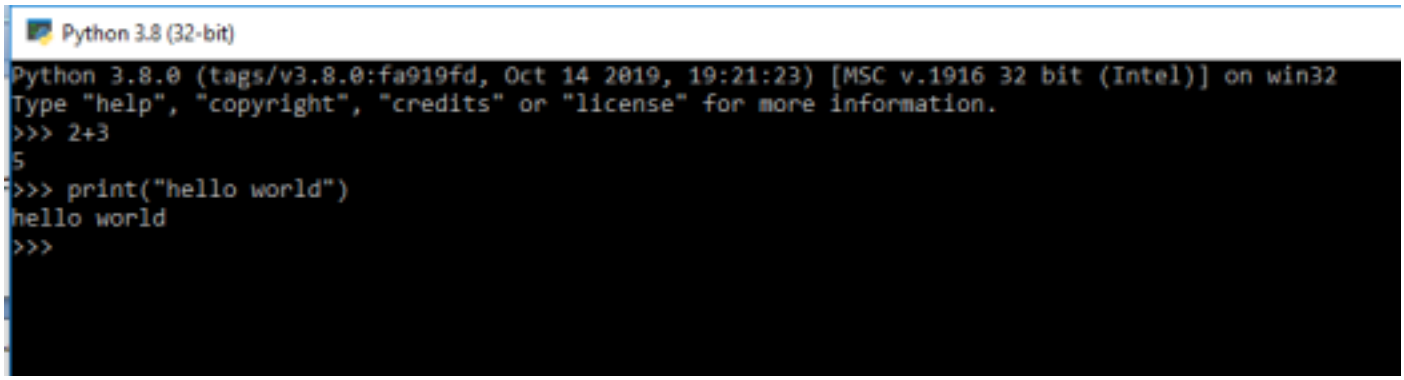
>>> x

#If a quantity is stored in memory, typing its name will display it.

[0, 1, 2]

>>> 2+3

5



The chevron at the beginning of the 1st line, i.e., the symbol >>> is a prompt the python interpreter uses to indicate that it is ready. If the programmer types 2+6, the interpreter replies 8.

Running Python in script mode:

Alternatively, programmers can store Python script source code in a file with the .py extension, and use the interpreter to execute the contents of the file. To execute the script by the interpreter, you have to tell the interpreter the name of the file. For example, if you have a script name MyFile.py and you're working on Unix, to run the script you have to type:

python MyFile.py

Working with the interactive mode is better when Python programmers deal with small pieces of code as you can type and execute them immediately, but when the code is more than 2-4 lines, using the script for coding can help to modify and use the code in future.

Example:

```
C:\Users\MRCET\AppData\Local\Programs\Python\Python38-32\pyyy>python e1.py
resource open
the no cant be divisible zero division by zero
resource close
finished
```

Data types:

The data stored in memory can be of many types. For example, a student roll number is stored as a numeric value and his or her address is stored as alphanumeric characters. Python has various standard data types that are used to define the operations possible on them and the storage method for each of them.

Int:

Int, or integer, is a whole number, positive or negative, without decimals, of unlimited length.

```
>>> print(24656354687654+2)
24656354687656
>>> print(20)
20
>>> print(0b10)
2
```



```
>>> print(0B10)
```

```
2
>>> print(0X20)
32
>>> 20
20
>>> 0b10
2
>>> a=10
>>> print(a)
10
```

To verify the type of any object in Python, use the type() function:

```
>>> type(10)
<class 'int'>
>>> a=11
>>> print(type(a))
<class 'int'>
```

Float:

Float, or "floating point number" is a number, positive or negative, containing one or more decimals.

Float can also be scientific numbers with an "e" to indicate the power of 10.

```
>>> y=2.8
>>> y
2.8
>>> y=2.8
>>> print(type(y))
<class 'float'>
>>> type(.4)
<class 'float'>
>>> 2.
```

2.0

Example:

```
x = 35e3
y = 12E4
z = -87.7e100

print(type(x))
print(type(y))
print(type(z))
```

Output:

```
<class 'float'>
<class 'float'>
<class 'float'>
```

Boolean:

Objects of Boolean type may have one of two values, True or False:

```
>>> type(True)

<class 'bool'>

>>> type(False)

<class 'bool'>
```

String:

1. Strings in Python are identified as a contiguous set of characters represented in the quotation marks. Python allows for either pairs of single or double quotes.

- 'hello' is the same as "hello".
- Strings can be output to screen using the print function. **For example: print("hello").**

```
>>> print("mrcet college")

mrcet college

>>> type("mrcet college")

<class 'str'>
```

```
>>> print('mrcet college')

mrcet college

>>> " "

' '
```

If you want to include either type of quote character within the string, the simplest way is to delimit the string with the other type. If a string is to contain a single quote, delimit it with double quotes and vice versa:

```
>>> print("mrcet is an autonomous (') college")

mrcet is an autonomous (') college

>>> print('mrcet is an autonomous (") college')

mrcet is an autonomous (") college
```

Suppressing Special Character:

Specifying a backslash (\) in front of the quote character in a string "escapes" it and causes Python to suppress its usual special meaning. It is then interpreted simply as a literal single quote character:

```
>>> print("mrcet is an autonomous (\') college")

mrcet is an autonomous (') college

>>> print('mrcet is an autonomous (\") college')

mrcet is an autonomous (") college
```

The following is a table of escape sequences which cause Python to suppress the usual special interpretation of a character in a string:

```
>>> print('a\
....b')
a ...b
>>> print('a\
b\
```

c')

abc

>>> print('a \n b')

a

b

>>> print("mrcet \n college")

mrcet

college

Escape Sequence	Usual Interpretation of Character(s) After Backslash	"Escaped" Interpretation
\'	Terminates string with single quote opening delimiter	Literal single quote (') character
\"	Terminates string with double quote opening delimiter	Literal double quote (") character
\newline	Terminates input line	Newline is ignored
\\	Introduces escape sequence	Literal backslash (\) character

In Python (and almost all other common computer languages), a tab character can be specified by the escape sequence \t:

>>> print("a\tb")

a b

List:

- It is a general purpose most widely used in data structures
- List is a collection which is ordered and changeable and allows duplicate members. (Grow and shrink as needed, sequence type, sortable).
- To use a list, you must declare it first. Do this using square brackets and separate values with commas.
- We can construct / create list in many ways.

Ex:

>>> list1=[1,2,3,'A','B',7,8,[10,11]]

```
>>> print(list1)
[1, 2, 3, 'A', 'B', 7, 8, [10, 11]]
```



```
>>> x=list()
>>> x
[]

>>> tuple1=(1,2,3,4)
>>> x=list(tuple1)
>>> x
[1, 2, 3, 4]
```

Variables:

Variables are nothing but reserved memory locations to store values. This means that when you create a variable you reserve some space in memory.

Based on the data type of a variable, the interpreter allocates memory and decides what can be stored in the reserved memory. Therefore, by assigning different data types to variables, you can store integers, decimals or characters in these variables.

Rules for Python variables:

- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (age, Age and AGE are three different variables)

Assigning Values to Variables:

Python variables do not need explicit declaration to reserve memory space. The declaration

happens automatically when you assign a value to a variable. The equal sign (=) is used to assign values to variables.

The operand to the left of the = operator is the name of the variable and the operand to the right of the = operator is the value stored in the variable.

For example −

a= 100 # An integer assignment

b = 1000.0 # A floating point

c = "John" # A string

print (a)

print (b)

print (c)

This produces the following result −

100

1000.0

John

Multiple Assignment:

Python allows you to assign a single value to several variables simultaneously.

For example :

a = b = c = 1

Here, an integer object is created with the value 1, and all three variables are assigned to the same memory location. You can also assign multiple objects to multiple variables.

For example −

a,b,c = 1,2,"mrcet“

Here, two integer objects with values 1 and 2 are assigned to variables a and b respectively,

and one string object with the value "john" is assigned to the variable c.

Output Variables:

The Python print statement is often used to output variables.

Variables do not need to be declared with any particular type and can even change type after they have been set.


```
x = 5 # x is of type int
x = "mrcet " # x is now of type str
print(x)
```

Output: mrcet

To combine both text and a variable, Python uses the **"+"** character:

Example

```
x = "awesome"
print("Python is " + x)
```

Output

Python is awesome

You can also use the + character to add a variable to another variable:

Example

```
x = "Python is "
y = "awesome"
z = x + y
print(z)
```

Output:

Python is awesome

Expressions:

An expression is a combination of values, variables, and operators. An expression is evaluated using assignment operator.

Examples: Y=x + 17

```
>>> x=10
>>> z=x+20
>>> z
30
```



```
>>> x=10
>>> y=20
>>> c=x+y
>>> c
30
```

A value all by itself is a simple expression, and so is a variable.

```
>>> y=20
>>> y
20
```

Python also defines expressions only contain identifiers, literals, and operators. So,

Identifiers: Any name that is used to define a class, function, variable module, or object is an identifier.

Literals: These are language-independent terms in Python and should exist independently in any programming language. In Python, there are the string literals, byte literals, integer literals, floating point literals, and imaginary literals.

Operators: In Python you can implement the following operations using the corresponding tokens.

Operator	Token
add	+
subtract	-
multiply	*
Integer Division	/
remainder	%
Binary left shift	<<
Binary right shift	>>
and	&
or	\
Less than	<

Greater than	>
Less than or equal to	<=
Greater than or equal to	>=
Check equality	==
Check not equal	!=

Some of the python expressions are:

Generator expression:

Syntax: (compute(var) for var in iterable)

```
>>> x = (i for i in 'abc') #tuple comprehension
>>> x
<generator object <genexpr> at 0x033EEC30>

>>> print(x)
<generator object <genexpr> at 0x033EEC30>
```

You might expect this to print as ('a', 'b', 'c') but it prints as <generator object <genexpr> at 0x02AAD710> The result of a tuple comprehension is not a tuple: it is actually a generator. The only thing that you need to know now about a generator now is that you can iterate over it, but ONLY ONCE.

Conditional expression:

Syntax: true_value if Condition else false_value

```
>>> x = "1" if True else "2"
```

>>> x

'1'

Statements:

A statement is an instruction that the Python interpreter can execute. We have normally two basic statements, the assignment statement and the print statement. Some other kinds of statements that are if statements, while statements, and for statements generally called as control flows.

Examples:

An assignment statement creates new variables and gives them values:

>>> x=10

>>> college="mrcet"

An print statement is something which is an input from the user, to be printed / displayed on to the screen (or) monitor.

>>> print("mrcet colege")

mrcet college

Precedence of Operators:

Operator precedence affects how an expression is evaluated.

For example, x = 7 + 3 * 2; here, x is assigned 13, not 20 because operator * has higher precedence than +, so it first multiplies 3*2 and then adds into 7.

Example 1:

>>> 3+4*2

11

Multiplication gets evaluated before the addition operation

>>> (10+10)*2

Parentheses () overriding the precedence of the arithmetic operators

Example 2:

```
a = 20
b = 10
c = 15
d = 5
e = 0

e = (a + b) * c / d #( 30 * 15 ) / 5
print("Value of (a + b) * c / d is ", e)

e = ((a + b) * c) / d # (30 * 15 ) / 5
print("Value of ((a + b) * c) / d is ", e)

e = (a + b) * (c / d); # (30) * (15/5)
```



```
print("Value of (a + b) * (c / d) is ", e)

e = a + (b * c) / d; # 20 + (150/5)
print("Value of a + (b * c) / d is ", e)
```

Output:

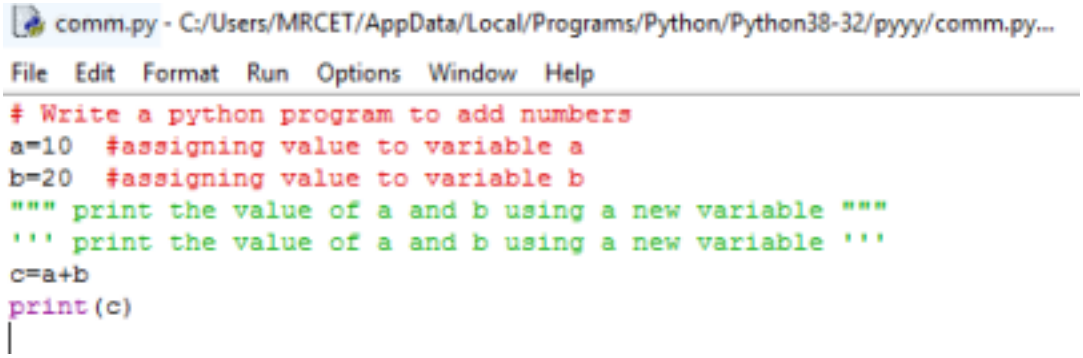
```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/opprec.p
y  Value of (a + b) * c / d is 90.0
Value of ((a + b) * c) / d is 90.0
Value of (a + b) * (c / d) is 90.0
Value of a + (b * c) / d is 50.0
```

Comments:

Single-line comments begins with a hash(#) symbol and is useful in mentioning that the whole line should be considered as a comment until the end of line.

A Multi line comment is useful when we need to comment on many lines. In python, triple double quote(“ “ “) and single quote(‘ ‘ ‘)are used for multi-line commenting.

Example:



Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/comm.p

y 30

Modules:

Modules: Python module can be defined as a python program file which contains a python code including python functions, class, or variables. In other words, we can say that our python code file saved with the extension (.py) is treated as the module. We may have a runnable code inside the python module. A module in Python provides us the flexibility to organize the code in a logical way. To use the functionality of one module into another, we must have to **import** the specific module.

Syntax:

import <module-name>

Every module has its own functions, those can be accessed with . (dot)

Note: In python we have help ()

Enter the name of any module, keyword, or topic to get help on writing Python programs and using Python modules. To quit this help utility and return to the interpreter, just type "quit".

Some of the modules like os, date, and calendar so on……

```
>>> import sys
>>> print(sys.version)
3.8.0 (tags/v3.8.0:fa919fd, Oct 14 2019, 19:21:23) [MSC v.1916 32 bit
(Intel)] >>> print(sys.version_info)
sys.version_info(major=3, minor=8, micro=0, releaselevel='final',
serial=0) >>> print(calendar.month(2021,5))
      May 2021
Mo Tu We Th Fr Sa Su
                1  2
 3  4  5  6  7  8  9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30
31
>>> print(calendar.isleap(2020))
True
>>> print(calendar.isleap(2017))
False
```


Functions:

Functions and its use: Function is a group of related statements that perform a specific task. Functions help break our program into smaller and modular chunks. As our program grows larger and larger, functions make it more organized and manageable. It avoids repetition and makes code reusable.

Basically, we can divide functions into the following two types:

1. **Built-in functions -** Functions that are built into Python.

   Ex: abs(),all().ascii(),bool()……so on….

   integer = -20

   print('Absolute value of -20 is:', abs(integer))

   Output:

   Absolute value of -20 is: 20

2. **User-defined functions -** Functions defined by the users themselves.

```
def add_numbers(x,y):
  sum = x + y
  return sum

print("The sum is", add_numbers(5, 20))
```

Output:

The sum is 25

Flow of Execution:

1. The order in which statements are executed is called the flow of execution
2. Execution always begins at the first statement of the program.
3. Statements are executed one at a time, in order, from top to bottom.
4. Function definitions do not alter the flow of execution of the program, but remember that statements inside the function are not executed until the function is called. 5. Function calls are like a bypass in the flow of execution. Instead of going to the next statement, the flow jumps to the first line of the called function, executes all the statements there, and then comes back to pick up where it left off.

Note: When you read a program, don't read from top to bottom. Instead, follow the flow of execution. This means that you will read the def statements as you are scanning from top to bottom, but you should skip the statements of the function definition until you reach a point where that function is called.

Example:

```
#example for flow of execution
print("welcome")
for x in range(3):
   print(x)
print("Good morning college")
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/flowof.p
y  welcome
```

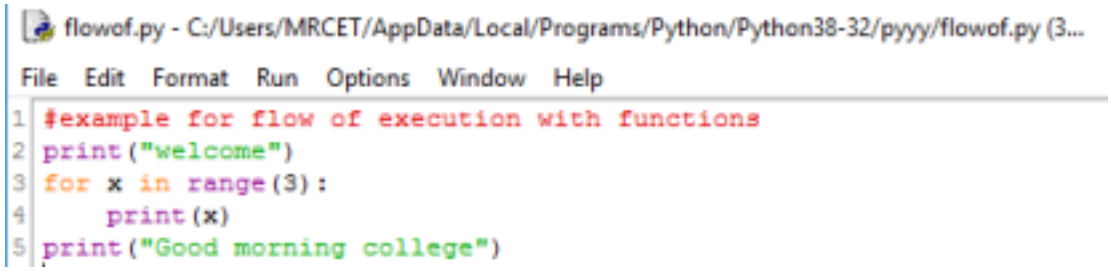
0

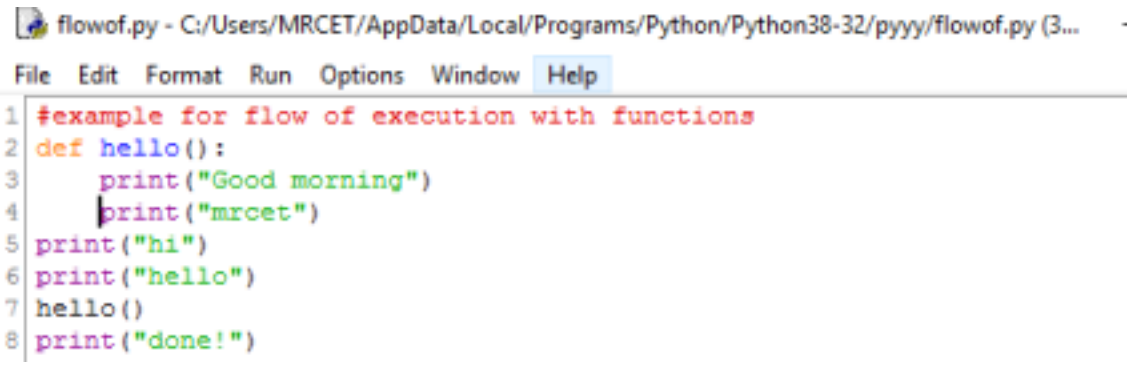
1

2

Good morning college

The flow/order of execution is: 2,3,4,3,4,3,4,5





Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/flowof.p

y hi

hello

Good morning

mrcet

done!

The flow/order of execution is: 2,5,6,7,2,3,4,7,8

Parameters and arguments:

Parameters are passed during the definition of function while Arguments are passed during the function call**.**

Example:

#here a and b are parameters

```
def add(a,b): #//function definition
 return a+b

#12 and 13 are arguments
#function call
result=add(12,13)
print(result)
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/paraarg.p

y 25

There are three types of Python function arguments using which we can call a function.

1. Default Arguments
2. Keyword Arguments
3. Variable-length Arguments

Syntax:

```
def functionname():
```



```
    statements
        .
        .
        .
functionname()
```

Function definition consists of following components:

1. Keyword **def** indicates the start of function header.
2. A function name to uniquely identify it. Function naming follows the same rules of writing identifiers in Python.
3. Parameters (arguments) through which we pass values to a function. They are optional.
4. A **colon (:)** to mark the end of function header.
5. Optional documentation string (docstring) to describe what the function does. 6. One or more valid python statements that make up the function body. Statements must have same indentation level (usually 4 spaces).
7. An optional return statement to return a value from the function.

Example:

```
def hf():
    hello world
hf()
```

In the above example we are just trying to execute the program by calling the function. So it will not display any error and no output on to the screen but gets executed.

To get the statements of function need to be use print().

#calling function in python:

```
def hf():
    print("hello world")
hf()
```

Output:

```
hello world
```



```
def hf():
    print("hw")
    print("gh kfjg 66666")
hf()
hf()
hf()
```

Output:

```
hw
gh kfjg 66666
hw
```

gh kfjg 66666
hw
gh kfjg 66666

```
def add(x,y):
    c=x+y
    print(c)
add(5,4)
```

Output:

9

```
def add(x,y):
    c=x+y
    return c
print(add(5,4))
```

Output:

9


```
def add_sub(x,y):
    c=x+y
    d=x-y
    return c,d
print(add_sub(10,5))
```

Output:

(15, 5)

The **return** statement is used to exit a function and go back to the place from where it was called. This statement can contain expression which gets evaluated and the value is returned. If there is no expression in the statement or the return statement itself is not present inside a function, then the function will return the **None** object.

```
def hf():
    return "hw"
print(hf())
```

Output:

hw

```
def hf():
    return "hw"
hf()
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/fu.p

y >>>


```
def hello_f():
    return "hellocollege"
print(hello_f().upper())
```

Output:

HELLOCOLLEGE

Passing Arguments

```
def hello(wish):
    return '{}'.format(wish)
print(hello("mrcet"))
```

Output:

mrcet

Here, the function wish() has two parameters. Since, we have called this function with two arguments, it runs smoothly and we do not get any error. If we call it with different number of arguments, the interpreter will give errors.

```
def wish(name,msg):
   """This function greets to
   the person with the provided message"""
       print("Hello",name + ' ' + msg)
wish("MRCET","Good morning!")
```

Output:

Hello MRCET Good morning!

Below is a call to this function with one and no arguments along with their respective error messages.

```
>>> wish("MRCET") # only one argument
TypeError: wish() missing 1 required positional argument: 'msg'
>>> wish() # no arguments
TypeError: wish() missing 2 required positional arguments: 'name' and 'msg'
```

```
def hello(wish,hello):
    return “hi” '{},{}'.format(wish,hello)
print(hello("mrcet","college"))
```

Output:

```
himrcet,college
```

#Keyword Arguments

When we call a function with some values, these values get assigned to the arguments according to their position.

Python allows functions to be called using keyword arguments. When we call functions in this way, the order (position) of the arguments can be changed.

(Or)

If you have some functions with many parameters and you want to specify only some of them, then you can give values for such parameters by naming them - this is called **keyword arguments** - we use the name (keyword) instead of the position (which we have been using all along) to specify the arguments to the function.

There are two *advantages* - one, using the function is easier since we do not need to worry about the order of the arguments. Two, we can give values to only those parameters which we want, provided that the other parameters have default argument values.

```
def func(a, b=5, c=10):
        print 'a is', a, 'and b is', b, 'and c is', c
```



```
func(3, 7)
func(25, c=24)
func(c=50, a=100)
```

Output:

a is 3 and b is 7 and c is 10
a is 25 and b is 5 and c is 24
a is 100 and b is 5 and c is 50

Note:

The function named func has one parameter without default argument values, followed by two parameters with default argument values.

In the first usage, func(3, 7), the parameter a gets the value 3, the parameter b gets the value 5 and c gets the default value of 10.

In the second usage func(25, c=24), the variable a gets the value of 25 due to the position of the argument. Then, the parameter c gets the value of 24 due to naming i.e. keyword arguments. The variable b gets the default value of 5.

In the third usage func(c=50, a=100), we use keyword arguments completely to specify the values. Notice, that we are specifying value for parameter c before that for a even though a is defined before c in the function definition.

For example: if you define the function like below

```
def func(b=5, c=10,a): # shows error : non-default argument follows default argument
```

```
def print_name(name1, name2):
  """ This function prints the name """
  print (name1 + " and " + name2 + " are friends")
#calling the function
print_name(name2 = 'A',name1 = 'B')
```


Output:

B and A are friends

#Default Arguments

Function arguments can have default values in Python.

We can provide a default value to an argument by using the assignment operator (=)

```
def hello(wish,name='you'):
    return '{},{}'.format(wish,name)
print(hello("good morning"))
```

Output:

good morning,you

```
def hello(wish,name='you'):
    return '{},{}'.format(wish,name) //print(wish + ‘ ‘ + name)
print(hello("good morning","nirosha")) // hello("good morning","nirosha")
```

Output:

good morning,nirosha // good morning nirosha

Note: Any number of arguments in a function can have a default value. But once we have a default argument, all the arguments to its right must also have default values.

This means to say, non-default arguments cannot follow default arguments. For example, if we had defined the function header above as:

```
def hello(name='you', wish):
```

Syntax Error: non-default argument follows default argument

```
def sum(a=4, b=2): #2 is supplied as default argument
```

```
  """ This function will print sum of two numbers
      if the arguments are not supplied
      it will add the default value """
  print (a+b)
sum(1,2) #calling with arguments
sum( ) #calling without arguments
```

Output:

3

6

Variable-length arguments

Sometimes you may need more arguments to process function then you mentioned in the definition. If we don't know in advance about the arguments needed in function, we can use variable-length arguments also called arbitrary arguments.

For this an asterisk (*) is placed before a parameter in function definition which can hold non-keyworded variable-length arguments and a double asterisk (**) is placed before a parameter in function which can hold keyworded variable-length arguments.

If we use one asterisk (*) like *var, then all the positional arguments from that point till the end are collected as a tuple called 'var' and if we use two asterisks (**) before a variable like **var, then all the positional arguments from that point till the end are collected as a dictionary called 'var'.

```
def wish(*names):
   """This function greets all
   the person in the names tuple."""

   # names is a tuple with arguments
   for name in names:
      print("Hello",name)

wish("MRCET","CSE","SIR","MADAM")
```

Output:

Hello MRCET

Hello CSE

Hello SIR

Hello MADAM

#Program to find area of a circle using function use single return value function with argument.

```
pi=3.14
def areaOfCircle(r):

    return pi*r*r
r=int(input("Enter radius of circle"))

print(areaOfCircle(r))
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/fu1.p
y  Enter radius of circle 3
28.259999999999998
```

#Program to write sum different product and using arguments with return value function.

```
def calculete(a,b):

    total=a+b

    diff=a-b

    prod=a*b

    div=a/b
```

```
    mod=a%b
```



```
    return total,diff,prod,div,mod

a=int(input("Enter a value"))

b=int(input("Enter b value"))

#function call

s,d,p,q,m = calculete(a,b)

print("Sum= ",s,"diff= ",d,"mul= ",p,"div= ",q,"mod= ",m)

#print("diff= ",d)

#print("mul= ",p)

#print("div= ",q)

#print("mod= ",m)
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/fu1.p
y  Enter a value 5
Enter b value 6
Sum= 11 diff= -1 mul= 30 div= 0.8333333333333334 mod= 5
```

#program to find biggest of two numbers using functions.

```
def biggest(a,b):
    if a>b :
        return a
    else :
        return b


a=int(input("Enter a value"))
b=int(input("Enter b value"))
#function call
```

```
big= biggest(a,b)
print("big number= ",big)
```

Output:


```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/fu1.p
y  Enter a value 5
Enter b value-2
big number= 5
```

#program to find biggest of two numbers using functions. (nested if)

```
def biggest(a,b,c):
    if a>b :
        if a>c :
            return a
        else :
            return c
    else :
        if b>c :
            return b
        else :
            return c


a=int(input("Enter a value"))
b=int(input("Enter b value"))
c=int(input("Enter c value"))
#function call
big= biggest(a,b,c)
print("big number= ",big)
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/fu1.p
y  Enter a value 5
Enter b value -6
Enter c value 7
big number= 7
```

#Writer a program to read one subject mark and print pass or fail use single return values function with argument.

```
def result(a):
    if a>40:
       return "pass"
```



```
    else:
        return "fail"
a=int(input("Enter one subject marks"))

print(result(a))
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/fu1.p
y  Enter one subject marks 35
fail
```

#Write a program to display mrecet cse dept 10 times on the screen. (while loop)

```
def usingFunctions():
    count =0
    while count<10:
        print("mrcet cse dept",count)
        count=count+1

usingFunctions()
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/fu1.p
y  mrcet cse dept 0
mrcet cse dept 1
mrcet cse dept 2
mrcet cse dept 3
mrcet cse dept 4
mrcet cse dept 5
mrcet cse dept 6
mrcet cse dept 7
mrcet cse dept 8
```

UNIT – II

CONTROL FLOW, LOOPS

Conditionals: Boolean values and operators, conditional (if), alternative (if-else), chained conditional (if-elif-else); Iteration: while, for, break, continue.

Control Flow, Loops:

Boolean Values and Operators:

A boolean expression is an expression that is either true or false. The following examples use the operator ==, which compares two operands and produces True if they are equal and False otherwise:

```
>>> 5 == 5
```

True

```
>>> 5 == 6
```

False

True and False are special values that belong to the type bool; they are not strings:

```
>>> type(True)
```

<class 'bool'>

```
>>> type(False)
```

<class 'bool'>

The == operator is one of the relational operators; the others are: x != y # x is not equal to y

x > y # x is greater than y x < y # x is less than y

x >= y # x is greater than or equal to y x <= y # x is less than or equal to y

Note:

All expressions involving relational and logical operators will evaluate to either true or false

Conditional (if):

The if statement contains a logical expression using which data is compared and a decision is made based on the result of the comparison.

Syntax:
if expression:
    statement(s)
If the boolean expression evaluates to TRUE, then the block of statement(s) inside the if statement is executed. If boolean expression evaluates to FALSE, then the first set of code after the end of the if statement(s) is executed.

if Statement Flowchart:

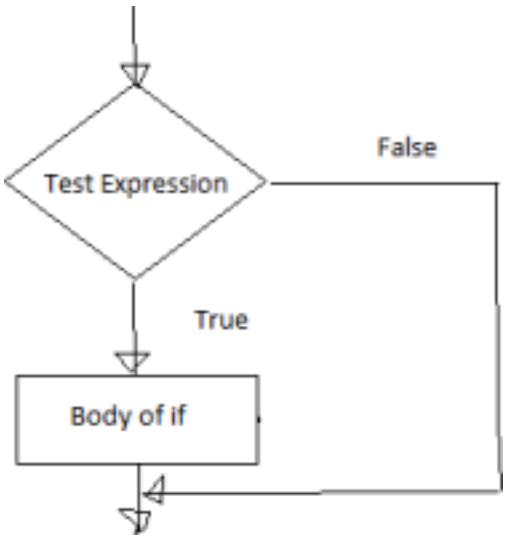


Fig: Operation of if statement

Example: Python if Statement

```
a = 3
if a > 2:
   print(a, "is greater")
print("done")

a = -1
if a < 0:
    print(a, "a is smaller")
print("Finish")
```

Output:


```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/if1.p
y  3 is greater
done
-1 a is smaller
Finish
```

```
a=10

if a>9:

   print("A is Greater than 9")
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/if2.p

y  A is Greater than 9
```

Alternative if (If-Else):

An else statement can be combined with an if statement. An else statement contains the block of code (false block) that executes if the conditional expression in the if statement resolves to 0 or a FALSE value.

The else statement is an optional statement and there could be at most only one else Statement following if.

Syntax of if - else :

```
if test expression:
        Body of if stmts
else:
        Body of else stmts
```

If - else Flowchart :

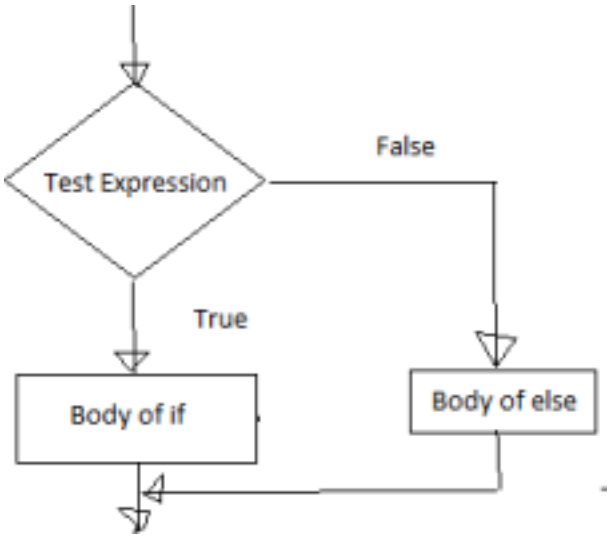


Fig: Operation of if – else statement

Example of if - else:

```
a=int(input('enter the number'))
if a>5:
   print("a is greater")
else:
   print("a is smaller than the input given")
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/ifelse.p
y  enter the number 2
```

a is smaller than the input given

```
a=10
b=20
if a>b:
   print("A is Greater than B")
else:
   print("B is Greater than A")
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/if2.p
y  B is Greater than A
```


Chained Conditional: (If-elif-else):

The elif statement allows us to check multiple expressions for TRUE and execute a block of code as soon as one of the conditions evaluates to TRUE. Similar to the else, the elif statement is optional. However, unlike else, for which there can be at most one statement, there can be an arbitrary number of elif statements following an if.

Syntax of if – elif - else :

```
If test expression:
        Body of ifstmts
elif test expression:
        Body of elif stmts
else:
        Body of else stmts
```

Flowchart of if – elif - else:

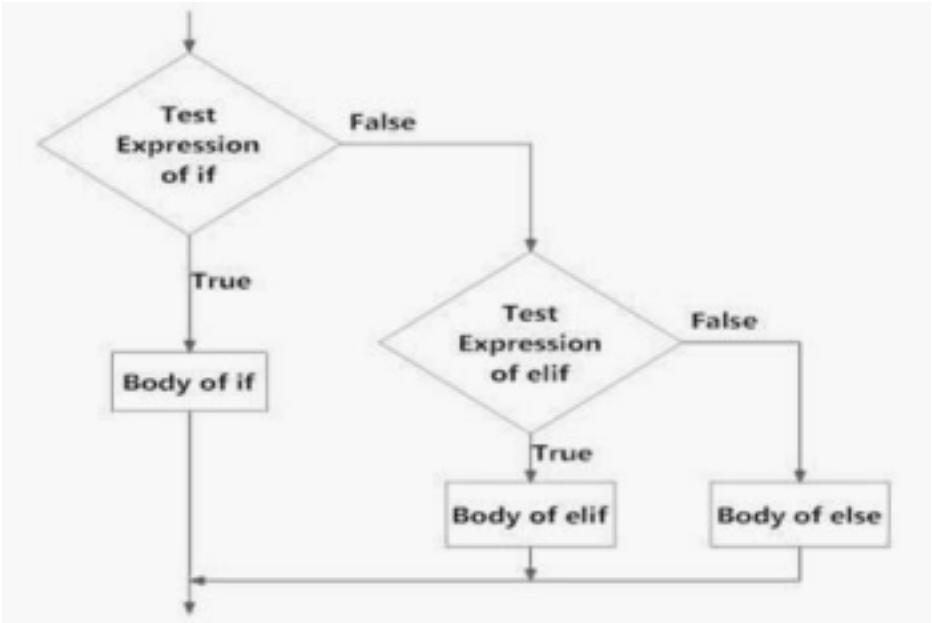


Fig: Operation of if – elif - else statement

Example of if - elif – else:

```
a=int(input('enter the number'))
b=int(input('enter the number'))
c=int(input('enter the number'))
if a>b:
```



```
    print("a is greater")
elif b>c:
    print("b is greater")
else:
    print("c is greater")
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/ifelse.p
y  enter the number5
enter the number2
enter the number9
a is greater
>>>
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/ifelse.p
y  enter the number2
enter the number5
enter the number9
c is greater
```

```
--------------------------
var = 100
if var == 200:
   print("1 - Got a true expression value")
   print(var)
elif var == 150:
   print("2 - Got a true expression value")
   print(var)
elif var == 100:
   print("3 - Got a true expression value")
   print(var)
else:
   print("4 - Got a false expression value")
   print(var)
print("Good bye!")
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/ifelif.p

y 3 - Got a true expression value

100

Good bye!

Iteration:

A loop statement allows us to execute a statement or group of statements multiple times as long as the condition is true. Repeated execution of a set of statements with the help of loops is called iteration.

Loops statements are used when we need to run same code again and again, each time with a different value.

Statements:

In Python Iteration (Loops) statements are of three types:

1. While Loop
2. For Loop
3. Nested For Loops

While loop:

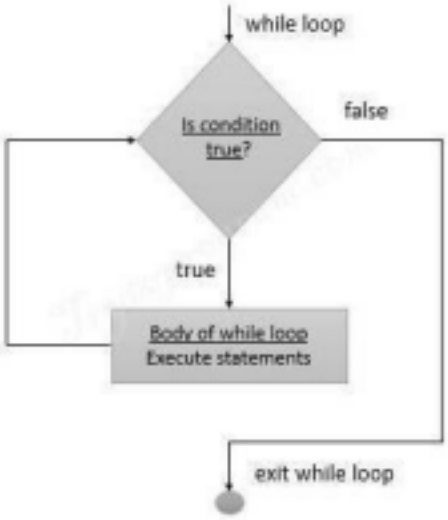
- Loops are either infinite or conditional. Python while loop keeps reiterating a block of code defined inside it until the desired condition is met.

• The while loop contains a boolean expression and the code inside the loop is repeatedly executed as long as the boolean expression is true.
• The statements that are executed inside while can be a single line of code or a block of multiple statements.

Syntax:

```
while(expression):
        Statement(s)
```

Flowchart:



Example Programs:

1.

```
i=1
while i<=6:
        print("Mrcet college")
        i=i+1
```

output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/wh1.p
y  Mrcet college
Mrcet college
Mrcet college
Mrcet college
```

Mrcet college

Mrcet college

2.

```
i=1

while i<=3:
    print("MRCET",end=" ")
    j=1
    while j<=1:
        print("CSE DEPT",end="")
        j=j+1
    i=i+1
    print()
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/wh2.p

y MRCET CSE DEPT

MRCET CSE DEPT

MRCET CSE DEPT

3.

```
i=1
```



```
j=1
while i<=3:
    print("MRCET",end=" ")

    while j<=1:
        print("CSE DEPT",end="")
        j=j+1
    i=i+1
    print()
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/wh3.py

MRCET CSE DEPT
MRCET

4.

```
i = 1
while (i < 10):
      print (i)
      i = i+1
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/wh4.p
y  1
2
3
4
5
6
7
8
9
```

2.

```
a = 1
b = 1
while (a<10):
    print ('Iteration',a)
    a = a + 1
    b = b + 1
```



```
        if (b == 4):
            break
    print ('While loop terminated')
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/wh5.p
y  Iteration 1
Iteration 2
Iteration 3
While loop terminated
```

```
count = 0
```

```
while (count < 9):
   print("The count is:", count)
   count = count + 1
print("Good bye!")
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/wh.py =
The count is: 0
The count is: 1
The count is: 2
The count is: 3
The count is: 4
The count is: 5
The count is: 6
The count is: 7
The count is: 8
Good bye!
```

For loop:

Python **for loop** is used for repeated execution of a group of statements for the desired number of times. It iterates over the items of lists, tuples, strings, the dictionaries and other iterable objects

Syntax: for var in sequence:

Statement(s) A sequence of values assigned to var in each iteration

Holds the value of item
in sequence in each iteration

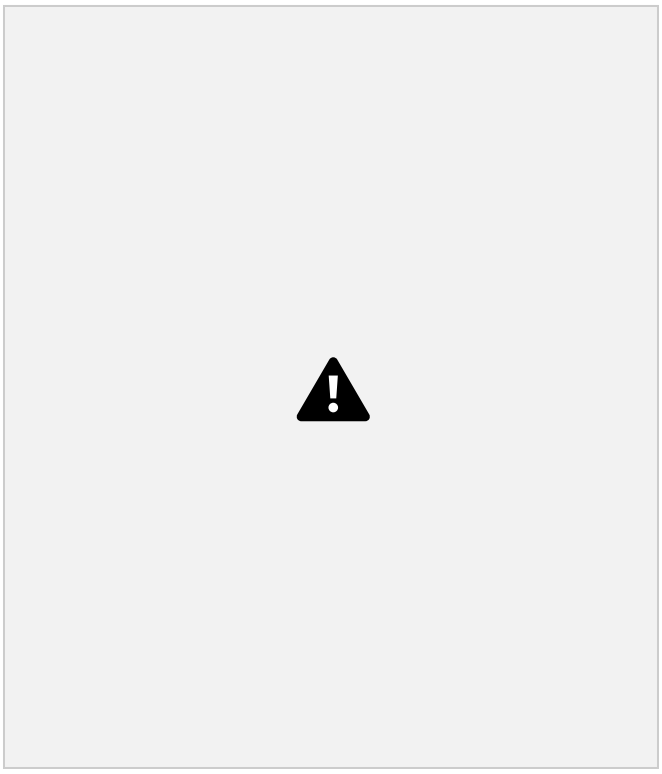
Sample Program:

```
numbers = [1, 2, 4, 6, 11, 20]
seq=0
for val in numbers:
   seq=val*val
   print(seq)
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/fr.py
1
4
16
36
121
400
```

Flowchart:



Iterating over a list:

```
#list of items
list = ['M','R','C','E','T']
i = 1

#Iterating over the list
for item in list:
  print ('college ',i,' is ',item)
  i = i+1
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/lis.p
y college 1 is M
college 2 is R
college 3 is C
college 4 is E
college 5 is T

Iterating over a Tuple:

```
tuple = (2,3,5,7)
print ('These are the first four prime numbers ')
#Iterating over the tuple
for a in tuple:
  print (a)
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/fr3.p
y These are the first four prime numbers
2
3
5
7

Iterating over a dictionary:

```
#creating a dictionary
college = {"ces":"block1","it":"block2","ece":"block3"}

#Iterating over the dictionary to print keys
print ('Keys are:')
```



```
for keys in college:
  print (keys)

#Iterating over the dictionary to print values
print ('Values are:')
for blocks in college.values():
  print(blocks)
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/dic.p

y Keys are:
ces
it
ece
Values are:
block1
block2
block3

Iterating over a String:

```
#declare a string to iterate over
college = 'MRCET'

#Iterating over the string
for name in college:
  print (name)
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/strr.py
M
R
C
E
T
```

Nested For loop:

When one Loop defined within another Loop is called Nested Loops. **Syntax:**

```
for val in sequence:
        for val in sequence:
            statements
    statements
```


Example 1 of Nested For Loops (Pattern Programs)

```
for i in range(1,6):
   for j in range(0,i):
```

```
    print(i, end=" ")
  print('')
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/nesforr.p
y  1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

Example 2 of Nested For Loops (Pattern Programs)

```
for i in range(1,6):
  for j in range(5,i-1,-1):
    print(i, end=" ")
  print('')
```

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/nesforr.py
```

Output:

```
1 1 1 1 1
2 2 2 2
3 3 3
4 4
```

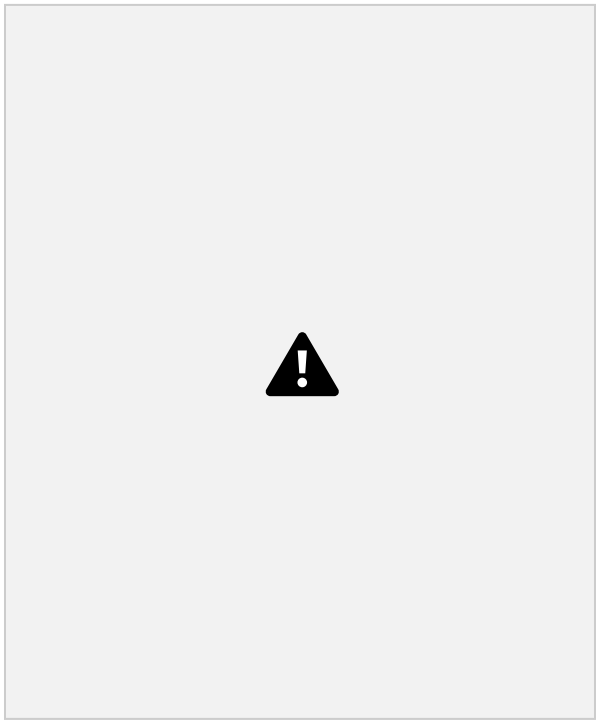

Break and continue:

In Python, **break and continue** statements can alter the flow of a normal loop. Sometimes we wish to terminate the current iteration or even the whole loop without checking test expression. The break and continue statements are used in these cases.

Break:

The break statement terminates the loop containing it and control of the program flows to the statement immediately after the body of the loop. If break statement is inside a nested loop (loop inside another loop), break will terminate the innermost loop.

Flowchart:



The following shows the working of break statement in for and while loop:

```
for var in sequence:
        # code inside for loop
        If condition:
                break (if break condition satisfies it jumps to outside loop)
        # code inside for loop
# code outside for loop
```



```
while test expression

        # code inside while loop
        If condition:
                break (if break condition satisfies it jumps to outside loop)
```

```
        # code inside while loop
# code outside while loop
```

Example:

```
for val in "MRCET COLLEGE":
    if val == " ":
        break
    print(val)

print("The end")
```

Output:

```
M
R
C
E
T
The end
```

Program to display all the elements before number 88

```
for num in [11, 9, 88, 10, 90, 3, 19]:
   print(num)
   if(num==88):
            print("The number 88 is found")
            print("Terminating the loop")
            break
```

Output:

```
11
9
88
The number 88 is found
```



```
Terminating the loop

#
for letter in "Python": # First Example
```

```
if letter == "h":
    break
print("Current Letter :", letter )
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/br.py =

Current Letter : P

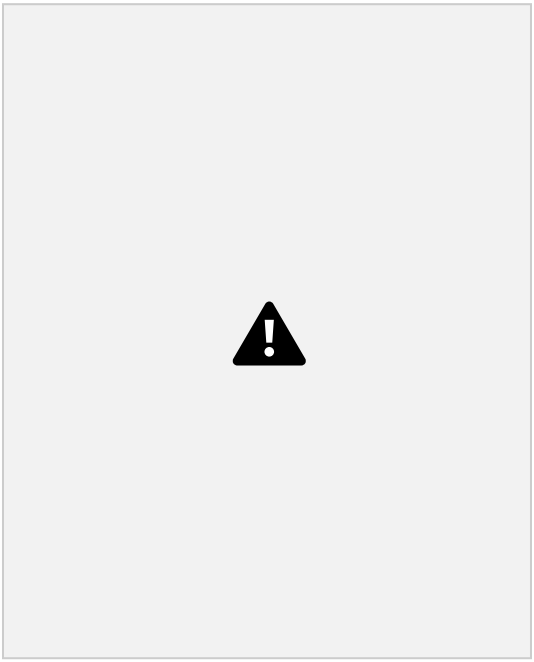
Current Letter : y

Current Letter : t

Continue:

The continue statement is used to skip the rest of the code inside a loop for the current iteration only. Loop does not terminate but continues on with the next iteration.

Flowchart:



The following shows the working of break statement in for and while loop:


```
for var in sequence:
        # code inside for loop
       If condition:
```

```
            continue (if break condition satisfies it jumps to outside loop)
        # code inside for loop
# code outside for loop


while test expression

        # code inside while loop
        If condition:
            continue(if break condition satisfies it jumps to outside loop)
        # code inside while loop
# code outside while loop
```

Example:

```
# Program to show the use of continue statement inside loops

for val in "string":
    if val == "i":
       continue
   print(val)

print("The end")
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/cont.p
y s
t
r
n
g
The end
```

```
# program to display only odd numbers

for num in [20, 11, 9, 66, 4, 89, 44]:
```



```
    # Skipping the iteration when number is even
    if num%2 == 0:
```

```
        continue
    # This statement will be skipped for all even numbers
    print(num)
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/cont2.p
y  11
9
89

#
for letter in "Python": # First Example
    if letter == "h":
        continue
    print("Current Letter :", letter)
Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/con1.p
y  Current Letter : P
Current Letter : y
Current Letter : t
Current Letter : o
Current Letter : n
```

Pass:

In Python programming, pass is a null statement. The difference between a comment and pass statement in Python is that, while the interpreter ignores a comment entirely, pass is not ignored.
pass is just a placeholder for functionality to be added later.

Example:

```
sequence = {'p', 'a', 's', 's'}
for val in sequence:
    pass
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/f1.y.py
```



```
>>>
```

Similarily we can also write,

def f(arg): pass # a function that does nothing (yet)

class C: pass # a class with no methods (yet)

UNIT – III

FUNCTIONS, ARRAYS

Fruitful functions: return values, parameters, local and global scope, function composition, recursion; Strings: string slices, immutability, string functions and methods, string module; Python arrays, Access the Elements of an Array, array methods.

Functions, Arrays:

Fruitful functions:

We write functions that return values, which we will call fruitful functions. We have seen the return statement before, but in a fruitful function the return statement includes a return value. This statement means: "Return immediately from this function and use the following expression as a return value."

(or)

Any function that returns a value is called Fruitful function. A Function that does not return a value is called a void function

Return values:

The Keyword return is used to return back the value to the called

function. **# returns the area of a circle with the given radius:**

```
def area(radius):
  temp = 3.14 * radius**2
return temp
print(area(4))
```

(or)

```
def area(radius):
  return 3.14 * radius**2
print(area(2))
```

Sometimes it is useful to have multiple return statements, one in each branch of a conditional:

```
def absolute_value(x):
    if x < 0:
```

```
        return -x
    else:
        return x
```

Since these return statements are in an alternative conditional, only one will be executed.

As soon as a return statement executes, the function terminates without executing any subsequent statements. Code that appears after a return statement, or any other place the flow of execution can never reach, is called dead code.

In a fruitful function, it is a good idea to ensure that every possible path through the program hits a return statement. For example:

```
def absolute_value(x):
    if x < 0:
        return -x
    if x > 0:
        return x
```

This function is incorrect because if x happens to be 0, both conditions is true, and the function ends without hitting a return statement. If the flow of execution gets to the end of a function, the return value is None, which is not the absolute value of 0.

```
>>> print absolute_value(0)
None
```

By the way, Python provides a built-in function called abs that computes absolute values.

Write a Python function that takes two lists and returns True if they have at least one common member.

```
def common_data(list1, list2):
    for x in list1:
        for y in list2:
            if x == y:
                result = True
                return result
print(common_data([1,2,3,4,5], [1,2,3,4,5]))
print(common_data([1,2,3,4,5], [1,7,8,9,510]))
print(common_data([1,2,3,4,5], [6,7,8,9,10]))
```

Output:

C:\Users\MRCET\AppData\Local\Programs\Python\Python38-32\pyyy\fu1.py

True
True
None

#

```
def area(radius):
    b = 3.14159 * radius**2
    return b
```

Parameters:

Parameters are passed during the definition of function while Arguments are passed during the function call**.**

Example:

```
#here a and b are parameters

def add(a,b): #//function definition
 return a+b

#12 and 13 are arguments
#function call
result=add(12,13)
print(result)
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/paraarg.p

y 25

Some examples on functions:

To display vandemataram by using function use no args no return type

```
#function defination
```

```
def display():
   print("vandemataram")
print("i am in main")
```



```
#function call
display()
print("i am in main")
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/fu1.p
y i am in main
vandemataram
i am in main
```

#Type1 : No parameters and no return type

```
def Fun1() :
   print("function 1")
Fun1()
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/fu1.p

y  function 1
```

#Type 2: with param with out return type

```
def fun2(a) :
    print(a)
fun2("hello")
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/fu1.p

y  Hello
```

#Type 3: without param with return type

```
def fun3():
   return "welcome to python"
print(fun3())
```


Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/fu1.p

y welcome to python

#Type 4: with param with return type

```
def fun4(a):
   return a
print(fun4("python is better then c"))
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/fu1.p

y python is better then c

Local and Global scope:

Local Scope:

A variable which is defined inside a function is local to that function. It is accessible from the point at which it is defined until the end of the function, and exists for as long as the function is executing

Global Scope:

A variable which is defined in the main body of a file is called a global variable. It will be visible throughout the file, and also inside any file which imports that file.

- The variable defined inside a function can also be made global by using the global statement.

```
def function_name(args):
```

.....

global x #declaring global variable inside a function

.....

create a global variable


```
x = "global"

def f():
    print("x inside :", x)

f()
print("x outside:", x)
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/fu1.p

y x inside : global

x outside: global

create a local variable

```
def f1():

    y = "local"

    print(y)

f1()
```

Output:

local

- If we try to access the local variable outside the scope for example,

```
def f2():
    y = "local"
```

```
f2()
print(y)
```

Then when we try to run it shows an error,

```
Traceback (most recent call last):
  File "C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/fu1.py", line
6, in <module>
```



```
    print(y)
NameError: name 'y' is not defined
```

The output shows an error, because we are trying to access a local variable y in a global scope whereas the local variable only works inside f2() or local scope.

use local and global variables in same code

```
x = "global"

def f3():
    global x
    y = "local"
    x = x * 2
    print(x)
    print(y)

f3()
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/fu1.p
y  globalglobal
local
```

- In the above code, we declare x as a global and y as a local variable in the f3(). Then, we use multiplication operator * to modify the global variable x and we print both x and y.
- After calling the f3(), the value of x becomes global global because we used the x * 2 to print two times global. After that, we print the value of local variable y i.e local.

use Global variable and Local variable with same name

```
x = 5

def f4():
    x = 10
    print("local x:", x)

f4()
print("global x:", x)
```


Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/fu1.p
y  local x: 10
global x: 5
```

Function Composition:

Having two (or more) functions where the output of one function is the input for another. So for example if you have two functions FunctionA and FunctionB you compose them by doing the following.

FunctionB(FunctionA(x))

Here x is the input for FunctionA and the result of that is the input for

FunctionB. **Example 1:**

#create a function compose2

```
>>> def compose2(f, g):
        return lambda x:f(g(x))

>>> def d(x):
        return x*2

>>> def e(x):
        return x+1

>>> a=compose2(d,e) # FunctionC = compose(FunctionB,FunctionA)
>>> a(5) # FunctionC(x)
```

In the above program we tried to compose n functions with the main function

created. **Example 2:**

```
>>> colors=('red','green','blue')

>>> fruits=['orange','banana','cherry']

>>> zip(colors,fruits)
```



```
<zip object at 0x03DAC6C8>

>>> list(zip(colors,fruits))

[('red', 'orange'), ('green', 'banana'), ('blue', 'cherry')]
```

Recursion:

Recursion is the process of defining something in terms of itself.

Python Recursive Function

We know that in Python, a function can call other functions. It is even possible for the function to call itself. These type of construct are termed as recursive functions.

Factorial of a number is the product of all the integers from 1 to that number. For example, the factorial of 6 (denoted as 6!) is 1*2*3*4*5*6 = 720.

Following is an example of recursive function to find the factorial of an integer.

```
# Write a program to factorial using recursion
def fact(x):
    if x==0:
        result = 1
    else :
        result = x * fact(x-1)
    return result
print("zero factorial",fact(0))
print("five factorial",fact(5))
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/rec.p
y  zero factorial 1
five factorial 120


def calc_factorial(x):
    """This is a recursive function
    to find the factorial of an integer"""

    if x == 1:
        return 1
    else:
        return (x * calc_factorial(x-1))
```



```
num = 4
print("The factorial of", num, "is", calc_factorial(num))
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/rec.p
y  The factorial of 4 is 24
```

Strings:

A string is a group/ a sequence of characters. Since Python has no provision for arrays, we simply use strings. This is how we declare a string. We can use a pair of single or double quotes. Every string object is of the type 'str'.

```
>>> type("name")
<class 'str'>
>>> name=str()
>>> name
''
>>> a=str('mrcet')
>>> a
'mrcet'
>>> a=str(mrcet)
>>> a[2]
'c'
>>> fruit = 'banana'
>>> letter = fruit[1]
```

The second statement selects character number 1 from fruit and assigns it to letter. The expression in brackets is called an index. The index indicates which character in the

sequence we want

String slices:
A segment of a string is called a slice. Selecting a slice is similar to selecting a character:

Subsets of strings can be taken using the slice operator **([ ] and [:])** with indexes starting at 0 in the beginning of the string and working their way from -1 at the end.

Slice out substrings, sub lists, sub Tuples using index.

Syntax:[Start: stop: steps]

- Slicing will start from index and will go up to **stop** in **step** of steps.
- Default value of start is 0,

- Stop is last index of list
- And for step default is 1

For example 1−

```
str = 'Hello World!'

print str # Prints complete string

print str[0] # Prints first character of the string print

str[2:5] # Prints characters starting from 3rd to 5th  print

str[2:] # Prints string starting from 3rd character print  str *

2 # Prints string two times

print str + "TEST" # Prints concatenated
```

string **Output:**

```
Hello World!

H

llo
```

llo World!

Hello World!Hello World!

Hello World!TEST

Example 2:

```
>>> x='computer'
>>> x[1:4]
'omp'
>>> x[1:6:2]
'opt'
>>> x[3:]
```



```
'puter'
>>> x[:5]
'compu'
>>> x[-1]
'r'
>>> x[-3:]
'ter'
>>> x[:-2]
'comput'
>>> x[::-2]
'rtpo'
>>> x[::-1]
'retupmoc'
```

Immutability:

It is tempting to use the [] operator on the left side of an assignment, with the intention of

changing a character in a string.
For example:

```
>>> greeting='mrcet college!'
>>> greeting[0]='n'
```

TypeError: 'str' object does not support item assignment

The reason for the error is that strings are **immutable**, which means we can't change an existing string. The best we can do is creating a new string that is a variation on the original:

```
>>> greeting = 'Hello, world!'
>>> new_greeting = 'J' + greeting[1:]
>>> new_greeting
'Jello, world!'
```

Note: The plus **(+)** sign is the string concatenation operator and the asterisk **(*)** is the repetition operator

String functions and methods:

There are many methods to operate on String.

S.no	Method name	Description
1.	isalnum()	Returns true if string has at least 1 character and all characters are alphanumeric and false otherwise.
2.	isalpha()	Returns true if string has at least 1 character and all characters are alphabetic and false otherwise.
3.	isdigit()	Returns true if string contains only digits and false otherwise.
4.	islower()	Returns true if string has at least 1 cased character and all cased characters are in lowercase and false otherwise.
5.	isnumeric()	Returns true if a string contains only numeric characters and false otherwise.

6.	isspace()	Returns true if string contains only whitespace characters and false otherwise.
7.	istitle()	Returns true if string is properly “titlecased” and false otherwise.
8.	isupper()	Returns true if string has at least one cased character and all cased characters are in uppercase and false otherwise.
9.	replace(old, new [, max])	Replaces all occurrences of old in string with new or at most max occurrences if max given.
	. split()	Splits string according to delimiter str (space if not provided) and returns list of substrings;
	. count()	Occurrence of a string in another string
	. find()	Finding the index of the first occurrence of a string in another string
	. swapcase()	Converts lowercase letters in a string to uppercase and viceversa
14.	startswith(str, beg=0,end=len(string))	Determines if string or a substring of string (if starting index beg and ending index end are given) starts with substring str; returns true if so and false otherwise.

Note:

All the string methods will be returning either true or false as the result

1. isalnum():

Isalnum() method returns true if string has at least 1 character and all characters are alphanumeric and false otherwise.

Syntax:

String.isalnum()

Example:

```
>>> string="123alpha"
>>> string.isalnum() True
```

2. isalpha():

isalpha() method returns true if string has at least 1 character and all characters are alphabetic and false otherwise.

Syntax:

String.isalpha()

Example:

```
>>> string="nikhil"
>>> string.isalpha()
True
```

3. isdigit():

isdigit() returns true if string contains only digits and false otherwise.

Syntax:

String.isdigit()

Example:

```
>>> string="123456789"
>>> string.isdigit()
True
```

4. islower():

Islower() returns true if string has characters that are in lowercase and false

otherwise. Syntax:

String.islower()

Example:

```
>>> string="nikhil"
>>> string.islower()
True
```

5. isnumeric():

isnumeric() method returns true if a string contains only numeric characters and false otherwise.

Syntax:

String.isnumeric()

Example:

```
>>> string="123456789"
>>> string.isnumeric()
True
```

6. isspace():

isspace() returns true if string contains only whitespace characters and false otherwise.

Syntax:

String.isspace()

Example:

```
>>> string=" "
>>> string.isspace()
True
```

7. istitle()

istitle() method returns true if string is properly “titlecased”(starting letter of each word is capital) and false otherwise

Syntax:

String.istitle()

Example:

```
>>> string="Nikhil Is Learning"
```

```
>>> string.istitle()
True
```

8. isupper()

isupper() returns true if string has characters that are in uppercase and false otherwise.

Syntax:

```
String.isupper()
```

Example:

```
>>> string="HELLO"
>>> string.isupper()
True
```

9. replace()

replace() method replaces all occurrences of old in string with new or at most max occurrences if max given.

Syntax:

```
String.replace()
```

Example:

```
>>> string="Nikhil Is Learning"
>>> string.replace('Nikhil','Neha')
 'Neha Is Learning'
```

10.split()

split() method splits the string according to delimiter str (space if not provided)

Syntax:

```
String.split()
```

Example:

```
>>> string="Nikhil Is Learning"
>>> string.split()
```



```
 ['Nikhil', 'Is', 'Learning']
```

11.count()

count() method counts the occurrence of a string in another string Syntax:

String.count()

Example:

```
>>> string='Nikhil Is Learning'
>>> string.count('i')
3
```

12.find()

Find() method is used for finding the index of the first occurrence of a string in another string

Syntax:

String.find(„string“)

Example:

```
>>> string="Nikhil Is Learning"
>>> string.find('k')
 2
```

13.swapcase()

converts lowercase letters in a string to uppercase and viceversa

Syntax:

String.find(„string“)

Example:

```
>>> string="HELLO"
>>> string.swapcase()
 'hello'
```

14.startswith()

Determines if string or a substring of string (if starting index beg and ending index end are given) starts with substring str; returns true if so and false otherwise.

Syntax:

String.startswith(„string“)

Example:

```
>>> string="Nikhil Is Learning"
>>> string.startswith('N')
True
```

15.endswith()

Determines if string or a substring of string (if starting index beg and ending index end are given) ends with substring str; returns true if so and false otherwise.

Syntax:

String.endswith(„string“)

Example:

```
>>> string="Nikhil Is Learning"
>>> string.startswith('g')
True
```

String module:

This module contains a number of functions to process standard Python strings. In recent versions, most functions are available as string methods as well.

It's a built-in module and we have to **import** it before using any of its constants and classes

Syntax: import string

Note:

help(string) --- gives the information about all the variables ,functions, attributes and classes to be used in string module.

Example:

```
import string
print(string.ascii_letters)
print(string.ascii_lowercase)
print(string.ascii_uppercase)
print(string.digits)
```

```
print(string.hexdigits)
#print(string.whitespace)
print(string.punctuation)
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/strrmodl.p
y ========================================
abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ
abcdefghijklmnopqrstuvwxyz
ABCDEFGHIJKLMNOPQRSTUVWXYZ
0123456789
0123456789abcdefABCDEF
!"#$%&'()*+,-./:;<=>?@[\]^_`{|}~
```

Python String Module Classes

Python string module contains two classes – Formatter and Template.

Formatter

It behaves exactly same as str.format() function. This class becomes useful if you want to subclass it and define your own format string syntax.

Syntax: from string import Formatter

Template

This class is used to create a string template for simpler string substitutions

Syntax: from string import Template

Python arrays:

Array is a container which can hold a fix number of items and these items should be of the same type. Most of the data structures make use of arrays to implement their algorithms. Following are the important terms to understand the concept of Array.

- **Element**− Each item stored in an array is called an element.
- **Index** − Each location of an element in an array has a numerical index, which is used to identify the element.

Array Representation

Arrays can be declared in various ways in different languages. Below is an illustration. Elements

Int array [10] = {10, 20, 30, 40, 50, 60, 70, 80, 85, 90}

Type Name Size Index 0

As per the above illustration, following are the important points to be considered. · Index starts with 0.

· Array length is 10 which means it can store 10 elements.

· Each element can be accessed via its index. For example, we can fetch an element at index 6 as 70

Basic Operations

Following are the basic operations supported by an array.

· Traverse − print all the array elements one by one.

· Insertion − Adds an element at the given index.

· Deletion − Deletes an element at the given index.

· Search − Searches an element using the given index or by the value.

· Update − Updates an element at the given index.

Array is created in Python by importing array module to the python program. Then the array is declared as shown below.

```
from array import *
arrayName=array(typecode, [initializers])
```

Typecode are the codes that are used to define the type of value the array will hold. Some common typecodes used are:

Typecode	Value
b	Represents signed integer of size 1 byte/td>

B	Represents unsigned integer of size 1 byte

c	Represents character of size 1 byte
i	Represents signed integer of size 2 bytes
l	Represents unsigned integer of size 2 bytes
f	Represents floating point of size 4 bytes
d	Represents floating point of size 8 bytes

Creating an array:

```
from array import *
array1 = array('i', [10,20,30,40,50])
for x in array1:
 print(x)
```

Output:

```
>>>

 RESTART: C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/arr.py

10
20
30
40
50
```

Access the elements of an Array:

Accessing Array Element

We can access each element of an array using the index of the element.

```
from array import *
array1 = array('i', [10,20,30,40,50])
print (array1[0])
print (array1[2])
```


Output:

```
RESTART: C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/arr2.py
10
30
```

Array methods:

Python has a set of built-in methods that you can use on lists/arrays.

Method	Description
append()	Adds an element at the end of the list
clear()	Removes all the elements from the list
copy()	Returns a copy of the list
count()	Returns the number of elements with the specified value
extend()	Add the elements of a list (or any iterable), to the end of the current list
index()	Returns the index of the first element with the specified value
insert()	Adds an element at the specified position
pop()	Removes the element at the specified position

remove()	Removes the first item with the specified value
reverse()	Reverses the order of the list
sort()	Sorts the list

Note: Python does not have built-in support for Arrays, but Python Lists can be used instead. **Example:**

```
>>> college=["mrcet","it","cse"]
>>> college.append("autonomous")
>>> college
['mrcet', 'it', 'cse', 'autonomous']
>>> college.append("eee")
>>> college.append("ece")
>>> college
['mrcet', 'it', 'cse', 'autonomous', 'eee', 'ece']
>>> college.pop()
'ece'
>>> college
['mrcet', 'it', 'cse', 'autonomous', 'eee']
>>> college.pop(4)
'eee'
>>> college
```

['mrcet', 'it', 'cse', 'autonomous']

>>> college.remove("it")

>>> college

['mrcet', 'cse', 'autonomous']

UNIT – IV

LISTS, TUPLES, DICTIONARIES

Lists: list operations, list slices, list methods, list loop, mutability, aliasing, cloning lists, list parameters, list comprehension; Tuples: tuple assignment, tuple as return value, tuple comprehension; Dictionaries: operations and methods, comprehension;

Lists, Tuples, Dictionaries:

List:

- It is a general purpose most widely used in data structures
- List is a collection which is ordered and changeable and allows duplicate members. (Grow and shrink as needed, sequence type, sortable).
- To use a list, you must declare it first. Do this using square brackets and separate values with commas.
- We can construct / create list in many ways.

Ex:

>>> list1=[1,2,3,'A','B',7,8,[10,11]]

>>> print(list1)

[1, 2, 3, 'A', 'B', 7, 8, [10, 11]]

>>> x=list()

>>> x

[]

```
>>> tuple1=(1,2,3,4)
>>> x=list(tuple1)
>>> x
[1, 2, 3, 4]
```


List operations:

These operations include indexing, slicing, adding, multiplying, and checking for membership

Basic List Operations:

Lists respond to the + and * operators much like strings; they mean concatenation and repetition here too, except that the result is a new list, not a string.

Python Expression	Results	Description
len([1, 2, 3])	3	Length
[1, 2, 3] + [4, 5, 6]	[1, 2, 3, 4, 5, 6]	Concatenation
['Hi!'] * 4	['Hi!', 'Hi!', 'Hi!', 'Hi!']	Repetition
3 in [1, 2, 3]	True	Membership
for x in [1, 2, 3]: print x,	1 2 3	Iteration

Indexing, Slicing, and Matrixes

Because lists are sequences, indexing and slicing work the same way for lists as they do for strings.

Assuming following input −

L = ['mrcet', 'college', 'MRCET!']

Python Expression	Results	Description
L[2]	MRCET	Offsets start at zero